Understanding Your StockPulse Backend — A Beginner's Guide

Why Does This Code Look So Different?

If you're used to writing Python like this:

```python
name = "Alice"
print("Hello " + name)

for i in range(10):
    print(i)
```

...then seeing things like `@dataclass`, `async def`, `self._client`, and `np.percentile()` can feel like a different language. **But it's all still Python!** It's just using **professional patterns** that you haven't encountered yet.

Think of it this way: you know how to write English sentences, but a legal contract also uses English — it just uses specialized vocabulary and structure. Same idea here.

This guide will walk you through **every "strange" thing** in your codebase, one concept at a time.

Part 1: The Big Picture — What Does This App Even Do?

Before diving into the code, let's understand the **purpose**:

StockPulse is a financial risk analysis backend that:

1. **Gets a list of S&P 500 stocks** (scrapes Wikipedia)
2. **Downloads daily stock prices** (from Polygon.io API)
3. **Stores everything in a database** (Supabase)
4. **Calculates "Value at Risk" (VaR)** — a measure of how much money you could lose on a stock at different confidence levels
5. **Runs this automatically every day** on weekdays

How the files are organized

```
stockpulse_backend/
│
├── main.py              ← The "front door" — you run this to start things
├── scheduler.py         ← Runs the pipeline automatically every day
├── requirements.txt     ← List of libraries to install
├── .env                 ← Secret keys (API keys, database passwords)
│
```

```
├── config/
│   └── settings.py        ← All the configuration/settings in one place
│
├── services/              ← "Workers" that each do one specific job
│   ├── supabase_client.py  ← Talks to the database
│   ├── polygon_client.py   ← Downloads stock prices from the internet
│   ├── var_engine.py       ← Does the math (calculates risk)
│   └── sp500_tracker.py    ← Gets the list of S&P 500 companies
│
├── jobs/                  ← "Tasks" that combine multiple services
│   ├── daily_pipeline.py   ← The main daily workflow (fetch + calculate)
│   └── backfill.py         ← Downloads historical data for new stocks
│
└── migrations/            ← SQL files to set up the database tables
```

Key insight: The code is organized into **layers**:

- `config/` = **Settings** (what to do)
- `services/` = **Tools** (how to do individual things)
- `jobs/` = **Workflows** (what order to do things in)
- `main.py` = **Entry point** (which workflow to run)

Part 2: New Python Concepts Explained

2.1 — Classes (the `class` keyword)

You probably write functions like this:

```python
# Simple approach you're used to:
def get_stock_price(ticker):
    # ... fetch price ...
    return price
```

But this codebase uses **classes**. A class is just a **bundle of related functions and data packaged together**.

```python
```

```python
class PolygonClient:
    def __init__(self, settings):
        self._api_key = settings.polygon_api_key    # Save the API key

    async def get_daily_bars(self, ticker, from_date, to_date):
        # Use self._api_key to fetch data
        ...
```

Why use a class? Imagine you need an API key for every function call. Instead of passing it every time:

```python
# Without classes — repetitive:
get_price("AAPL", api_key="abc123")
get_history("AAPL", api_key="abc123")
get_bars("AAPL", api_key="abc123")
```

You store it once in the class:

```python
# With classes — set it once, use it everywhere:
client = PolygonClient(settings)  # API key saved inside
client.get_daily_bars("AAPL", ...)  # Already knows the key!
```

What is `self`?

`self` is just the class referring to **itself**. When you write `self._api_key`, you're saying "save this on me, so I can use it later."

```python
class Dog:
    def __init__(self, name):
        self.name = name        # "Remember my name"

    def bark(self):
        print(f"{self.name} says woof!")  # "Use my name"

rex = Dog("Rex")
rex.bark()  # Prints: "Rex says woof!"
```

Every method (function inside a class) gets `self` as its first parameter — it's how the method knows which specific object it belongs to.

What is `__init__`?

`__init__` is the **constructor** — it runs automatically when you create a new object. It's where you set up the initial state.

```python
class SupabaseClient:
    def __init__(self, settings):
        # This runs when you write: client = SupabaseClient(settings)
        self._client = create_client(settings.supabase_url, settings.supabase_service_key)
```

What does the underscore `_` mean? (e.g., `self._client`)

A single underscore prefix is a **convention** meaning "this is private — don't touch it from outside." It's like a "staff only" sign. Python doesn't enforce this, but it signals intent:

```python
class PolygonClient:
    def __init__(self, settings):
        self._api_key = settings.polygon_api_key   # "Private" — internal use only
        self._delay = settings.price_fetch_delay_sec
```

2.2 — Type Hints

You'll see things like this everywhere:

```python
def get_latest_price_date(self, ticker: str) -> date | None:
```

This is NOT a new syntax that changes how Python works. It's just **a label** telling readers (and tools) what types the function expects and returns:

- `ticker: str` → "ticker should be a string"
- `-> date | None` → "this function returns either a date or None"
- `-> list[dict]` → "this function returns a list of dictionaries"

Python ignores these completely at runtime. They're just documentation. Think of them like comments that follow a standardized format.

```python
```

```python
# These two are identical in behavior:
def greet(name: str) -> str:
    return f"Hello {name}"

def greet(name):
    return f"Hello {name}"
```

2.3 — `@dataclass` (The Settings File)

In `config/settings.py`, you'll see:

```python
from dataclasses import dataclass, field

@dataclass
class Settings:
    supabase_url: str = field(default_factory=lambda: os.environ["SUPABASE_URL"])
    test_mode: bool = field(default_factory=lambda: os.environ.get("TEST_MODE", "true").lower() == "true")
```

The `@dataclass` decorator is a shortcut. Normally, to make a class that holds data, you'd write:

```python
# WITHOUT @dataclass — lots of boilerplate:
class Settings:
    def __init__(self):
        self.supabase_url = os.environ["SUPABASE_URL"]
        self.test_mode = os.environ.get("TEST_MODE", "true").lower() == "true"
```

```python
# WITH @dataclass — Python generates __init__ for you:
@dataclass
class Settings:
    supabase_url: str = field(default_factory=lambda: os.environ["SUPABASE_URL"])
    test_mode: bool = field(default_factory=lambda: os.environ.get("TEST_MODE", "true").lower() == "true")
```

Both do the same thing! `@dataclass` just saves you from writing `__init__` yourself.

What's `field(default_factory=lambda: ...)`?

This is the trickiest part. Let's break it down:

- `field(...)` = "Here's how to create the default value for this field"

- `default_factory=` = "Call this function to get the default value"
- `lambda:` = "Here's a tiny anonymous function"
- `os.environ["SUPABASE_URL"]` = "Read this from environment variables"

Why not just write `supabase_url: str = os.environ["SUPABASE_URL"]`?

Because that would read the environment variable **when the file is first loaded**, not when you actually create a Settings object. Using `default_factory` delays it until you actually write `settings = Settings()`.

What's `@property`?

```python
@property
def var_confidence_step(self) -> float:
    return 0.01 if self.test_mode else 0.001
```

`@property` lets you call a method **as if it were a simple attribute**:

```python
# Without @property:
settings.var_confidence_step()   # Need parentheses ()

# With @property:
settings.var_confidence_step     # No parentheses — looks like a simple value!
```

It's useful when a value needs to be **computed** but you want it to **look simple** to the caller.

2.4 — `async` and `await` (The Biggest "What?!")

This is probably the #1 reason the code looks unfamiliar. You'll see:

```python
async def main():
    await pipeline.run()

asyncio.run(main())
```

The problem async solves

Imagine you need to download stock prices for 500 stocks, and each download takes 12 seconds. Normally:

```python
```

```
# Synchronous (blocking) — what you're used to:
for ticker in tickers:
    prices = download_prices(ticker)  # Waits 12 seconds
    save(prices)
# Total: 500 × 12 seconds = 100 minutes of WAITING
```

During those 12 seconds of waiting for each download, your program is doing **nothing**. It's just sitting there.

`async` lets your program do other work while waiting:

```python
# Async (non-blocking):
async def download_prices(ticker):
    prices = await http_client.get(url)  # "Start downloading, I'll do other stuff"
    return prices
```

The three keywords to remember

Keyword	What it means	Analogy
`async def`	"This function might need to wait for something"	Ordering food at a restaurant
`await`	"Wait for this to finish, but let others work meanwhile"	"I'll wait for my food but others can order"
`asyncio.run()`	"Start the async system"	Opening the restaurant

Rules of async

1. If a function uses `await`, it MUST be defined with `async def`
2. You can only use `await` inside an `async def` function
3. To run the first async function, you use `asyncio.run()`

```python
# This is the pattern you see in main.py:
async def main():                    # 1. Define as async
    bars = await polygon.get_daily_bars(...)  # 2. await inside async

if __name__ == "__main__":
    asyncio.run(main())              # 3. Start the async system
```

What's a Semaphore?

In `polygon_client.py`:

```python
self._semaphore = asyncio.Semaphore(1)

async with self._semaphore:
    # Only 1 request at a time can run here
```

A semaphore is like a **bouncer at a club**: "Only 1 person allowed in at a time." This prevents making too many API calls at once (which would get you rate-limited/blocked).

2.5 — `if __name__ == "__main__":`

You'll see this at the bottom of `main.py` and `scheduler.py`:

```python
if __name__ == "__main__":
    asyncio.run(main())
```

This means: **"Only run this code if this file is being run directly."**

- If you run `python main.py` → `__name__` is `"__main__"` → the code runs
- If another file does `from main import something` → `__name__` is `"main"` → the code does NOT run

It's a safety guard so that importing the file doesn't accidentally start the whole program.

2.6 — Environment Variables and `.env`

```python
os.environ["SUPABASE_URL"]
```

Environment variables are **secret settings stored outside your code**. You never put passwords or API keys directly in your code (because if you share the code, everyone sees your passwords).

Instead, you put them in a `.env` file:

```
SUPABASE_URL=https://xyz.supabase.co
SUPABASE_SERVICE_ROLE_KEY=super-secret-key
POLYGON_API_KEY=another-secret-key
```

Then this line loads them into Python:

```python
```

```
from dotenv import load_dotenv
load_dotenv()  # Reads .env file and puts values into os.environ
```

Now `os.environ["SUPABASE_URL"]` gives you `"https://xyz.supabase.co"`.

2.7 — Logging (instead of `print()`)

Instead of `print("something happened")`, professional code uses **logging**:

```python
import logging
logger = logging.getLogger(__name__)

logger.info("Processing started")      # Normal info
logger.warning("Not enough data")      # Something concerning
logger.error("API call failed")        # Something broke
```

Why not just use `print()`?

- Logging adds **timestamps** automatically: `2025-01-15 10:30:00 [INFO] Processing started`
- Logging has **levels** (DEBUG, INFO, WARNING, ERROR) — you can filter what you see
- Logging can write to **files**, not just the screen
- You can turn logging on/off per module without changing any code

---

2.8 — `argparse` (Command-Line Arguments)

In `main.py`:

```python
parser = argparse.ArgumentParser(description="StockPulse VaR Pipeline")
group = parser.add_mutually_exclusive_group(required=True)
group.add_argument("--run-daily", action="store_true")
group.add_argument("--backfill", type=str, metavar="TICKER")
```

This lets you run the program with different commands:

```bash
python main.py --run-daily          # Run the daily pipeline
python main.py --backfill AAPL      # Backfill just Apple
python main.py --describe           # Show the current config
```

`add_mutually_exclusive_group` means you can only pick ONE option — you can't run `--daily` and `--backfill` at the same time.

2.9 — NumPy (`np`)

In `var_engine.py`:

```python
import numpy as np

closes = np.array([150.0, 151.5, 149.0, ...], dtype=np.float64)
log_returns = np.diff(np.log(closes))
percentiles = np.percentile(log_returns, [5, 10, 95])
```

NumPy is a math library that makes working with large lists of numbers **much faster**. Here's what each function does:

Code	What it does	Simple equivalent
`np.array([1,2,3])`	Creates a fast number list	Same as `[1,2,3]` but faster for math
`np.log(closes)`	Takes the natural log of every number	`[math.log(x) for x in closes]`
`np.diff(arr)`	Gets the difference between consecutive items	`[arr[i+1]-arr[i] for i in range(len(arr)-1)]`
`np.percentile(arr, 5)`	Finds the 5th percentile value	(Complex stats calculation)

Part 3: Walking Through Each File

3.1 — `config/settings.py` — The Control Panel

Purpose: One central place for ALL settings. Nothing is hardcoded in other files.

```python
```

```python
@dataclass
class Settings:
    # Database connection info (from .env file)
    supabase_url: str = field(default_factory=lambda: os.environ["SUPABASE_URL"])

    # Test mode: only process 10 stocks instead of 500
    test_mode: bool = True

    # The 10 test stocks
    test_tickers: tuple = ("AAPL", "NVDA", "TSLA", ...)

    # How many years of data to analyze (252 trading days = 1 year)
    var_lookback_windows: tuple = (252, 504, 756, 1008, 1260)
```

Key idea: By changing `test_mode` to `False`, the entire app switches from processing 10 stocks to 500+ stocks. No other file needs to change.

3.2 — `services/supabase_client.py` — The Database Talker

Purpose: Every database operation goes through this file. No other file talks to Supabase directly.

```python
class SupabaseClient:
    def __init__(self, settings):
        self._client = create_client(settings.supabase_url, settings.supabase_service_key)

    def get_sp500_tickers(self) -> list[dict]:
        resp = self._client.table("sp500_constituents").select("*").execute()
        return resp.data
```

The Supabase library uses **method chaining** — stacking methods one after another:

```python
resp = (
    self._client.table("price_history")   # 1. Go to the "price_history" table
    .select("date, close, adj_close")      # 2. I only want these columns
    .eq("ticker", ticker)                  # 3. Where ticker = "AAPL"
    .order("date", desc=True)              # 4. Newest dates first
    .limit(1)                              # 5. Give me just 1 row
    .execute()                             # 6. Actually run the query!
)
```

This is equivalent to SQL: `SELECT date, close, adj_close FROM price_history WHERE ticker = 'AAPL' ORDER BY date DESC LIMIT 1`

What is "upsert"? It means "**up**date or in**sert**": if the row exists, update it. If it doesn't exist, insert it. This prevents duplicate data.

3.3 — `services/polygon_client.py` — The Stock Price Downloader

Purpose: Downloads stock price data from the Polygon.io API.

```python
class PolygonClient:
    async def get_daily_bars(self, ticker, from_date, to_date) -> list[dict]:
        url = f"{POLYGON_BASE}/v2/aggs/ticker/{ticker}/range/1/day/{from_date}/{to_date}"

        async with httpx.AsyncClient(timeout=30) as client:
            resp = await client.get(url, params=params)
            data = resp.json()

            for bar in data.get("results", []):
                # Convert each "bar" (day of trading) into a clean dictionary
                all_rows.append({
                    "ticker": ticker,
                    "date": bar_date.isoformat(),
                    "close": bar.get("c"),
                    ...
                })

        await asyncio.sleep(self._delay)  # Wait 12.5 seconds (rate limit)
```

Key concepts here:

1. `httpx` is like the `requests` library but supports async. It makes HTTP calls to APIs.
2. `async with` means "open this connection, and make sure to close it properly when done."
3. `bar.get("c")` — Polygon returns short field names (`c` for close, `o` for open, `h` for high, `l` for low). `.get()` safely returns `None` if the key is missing instead of crashing.
4. `await asyncio.sleep(12.5)` — Waits 12.5 seconds between API calls to avoid hitting Polygon's rate limit (free tier: 5 calls/minute).

3.4 — `services/var_engine.py` — The Math Brain

Purpose: Calculates Value at Risk using "historical simulation" — looking at actual past price movements to

estimate future risk.

```python
class VaREngine:
    def compute_var_for_ticker(self, ticker, prices, calc_date, confidence_levels, lookback_windows):
        # Step 1: Get closing prices as a numpy array
        closes = np.array([float(p["adj_close"] or p["close"]) for p in prices])

        # Step 2: Calculate daily log returns
        log_returns = np.diff(np.log(closes))

        # Step 3: For each lookback window (1yr, 2yr, etc.)
        for lookback in lookback_windows:
            window_returns = log_returns[-lookback:]  # Last N days

            # Step 4: For each confidence level (80%, 81%, ... 99%)
            # Find the percentile of returns
            lower_pcts = np.percentile(window_returns, lower_quantiles)

            # Step 5: Convert percentage to dollar amount
            var_lower = reference_price * math.exp(var_pct_lower)
```

What this is doing in plain English:

1. Take 5 years of closing prices for Apple
2. Calculate the daily % change for each day
3. Look at the worst days (e.g., "95% of the time, Apple doesn't drop more than X%")
4. Convert that percentage into a dollar amount ("At 95% confidence, Apple won't drop below $180 tomorrow")

What's a "log return"? `np.log(price_today / price_yesterday)` — it's the standard way finance calculates returns because it has nice mathematical properties (like being additive over time).

3.5 — `services/sp500_tracker.py` — The Stock List Scraper

Purpose: Scrapes Wikipedia to get the current list of S&P 500 companies.

```python
```

```python
class SP500Tracker:
    async def _scrape_wikipedia(self) -> list[dict]:
        async with httpx.AsyncClient(timeout=20) as client:
            resp = await client.get(WIKIPEDIA_URL)

        soup = BeautifulSoup(resp.text, "html.parser")
        table = soup.find("table", {"id": "constituents"})

        rows = table.find_all("tr")[1:]   # Skip the header row
        for row in rows:
            cols = row.find_all("td")
            ticker = cols[0].get_text(strip=True)
            # ... build the constituent dict
```

BeautifulSoup is a library for parsing HTML. It turns a messy HTML page into something you can navigate like a tree:

- `soup.find("table")` → finds the first `<table>` element
- `table.find_all("tr")` → finds all `<tr>` (table row) elements
- `cols[0].get_text(strip=True)` → gets the text from the first column, removing whitespace

---

3.6 — `jobs/daily_pipeline.py` — The Conductor

Purpose: Orchestrates the entire daily process in order. This is like a recipe — it tells the services what to do and in what order.

```python
```

```python
class DailyPipeline:
    async def run(self):
        # Step 1: Update the list of S&P 500 companies
        diff = await self.sp500.refresh()

        # Step 2: Decide which stocks to process
        tickers = self._resolve_tickers(diff["added"])

        # Step 3: Backfill any new stocks that have no data
        for ticker in tickers:
            latest = self.supabase.get_latest_price_date(ticker)
            if latest is None:
                await backfill.backfill_ticker(ticker)

        # Step 4: Download today's prices
        await self._fetch_daily_prices(tickers, today)

        # Step 5: Calculate VaR for all stocks
        await self._compute_var_all(tickers, today)
```

Notice how clean this is? Each step calls a service. The pipeline doesn't know HOW to scrape Wikipedia or calculate VaR — it just tells the right service to do it.

3.7 — `main.py` — The Front Door

Purpose: Parses your command-line arguments and routes to the right job.

This is the **entry point** — the file you actually run. It:

1. Reads which command you typed (`--run-daily`, `--backfill`, etc.)
2. Creates all the service objects
3. Runs the appropriate job

```python
```

```python
async def main():
    # Parse command-line arguments
    args = parser.parse_args()

    # Create all the services
    settings = Settings()
    supabase = SupabaseClient(settings)
    polygon = PolygonClient(settings)
    sp500 = SP500Tracker(supabase)
    var_engine = VaREngine()

    # Route to the right job
    if args.run_daily:
        await pipeline.run()
    elif args.backfill:
        await backfill.backfill_ticker(args.backfill)
```

3.8 — `scheduler.py` — The Alarm Clock

Purpose: Runs the daily pipeline automatically at 9:30 PM UTC every weekday.

```python
scheduler.add_job(
    run_pipeline,
    trigger=CronTrigger(day_of_week="mon-fri", hour=21, minute=30, timezone="UTC"),
)
```

Why 21:30 UTC? That's after US stock markets close (4 PM ET = 21:00 UTC), with 30 minutes buffer for data to be available.

What's a "cron trigger"? Cron is a scheduling system from Unix. `day_of_week="mon-fri"` + `hour=21, minute=30` means "every weekday at 9:30 PM UTC."

Part 4: Design Patterns Used

4.1 — Dependency Injection

Instead of each class creating its own dependencies:

```python
```

```
# BAD: hard to test, hard to change
class DailyPipeline:
    def __init__(self):
        self.supabase = SupabaseClient(Settings())  # Creates its own
        self.polygon = PolygonClient(Settings())    # Creates its own
```

Dependencies are **passed in from outside**:

```python
# GOOD: flexible, testable
class DailyPipeline:
    def __init__(self, settings, supabase, polygon, sp500_tracker, var_engine):
        self.settings = settings
        self.supabase = supabase      # Received from outside
        self.polygon = polygon        # Received from outside
```

Why? You could swap `SupabaseClient` with a `FakeDatabase` for testing without changing `DailyPipeline` at all.

4.2 — Separation of Concerns

Each file does **one thing**:

- `polygon_client.py` → talks to Polygon API (nothing else)
- `supabase_client.py` → talks to the database (nothing else)
- `var_engine.py` → does math (nothing else)

No file reaches into another file's job. The `VaREngine` doesn't know where prices come from. The `PolygonClient` doesn't know what happens to the data after it's downloaded.

4.3 — Configuration Separation

All magic numbers and settings live in `settings.py`, not scattered across files. Want to change the rate limit? Change ONE number in ONE file. Want to switch from test to production? Flip ONE boolean.

Part 5: How Data Flows Through the System

```
main.py (--run-daily)
    │
    ▼
DailyPipeline.run()
    │
    ├── 1. SP500Tracker.refresh()
    │       └── Scrapes Wikipedia → Upserts to Supabase "sp500_constituents" table
    │
```

```
├── 2. _resolve_tickers()
│       └── Returns 10 tickers (test) or ~500 (production)
│
├── 3. BackfillJob (for new tickers only)
│       ├── PolygonClient.get_daily_bars() → downloads years of prices
│       ├── SupabaseClient.upsert_price_history() → saves to DB
│       └── VaREngine.compute_var_for_ticker() → calculates risk
│
├── 4. _fetch_daily_prices()
│       ├── PolygonClient.get_daily_bars() → downloads last 5 days
│       └── SupabaseClient.upsert_price_history() → saves to DB
│
└── 5. _compute_var_all()
        ├── SupabaseClient.get_price_history() → reads prices from DB
        ├── VaREngine.compute_var_for_ticker() → calculates risk
        └── SupabaseClient.upsert_var_calculations() → saves VaR to DB
```

Part 6: Quick Reference — "What Does This Syntax Mean?"

Syntax	Meaning	Example
`class Foo:`	Bundle of related functions and data	`class PolygonClient:`
`def __init__(self):`	Setup code that runs when you create an object	Constructor
`self.x`	Data stored on this object	`self._api_key`
`self._x`	"Private" data (convention, not enforced)	`self._client`
`async def`	Function that can pause and wait	`async def get_daily_bars(...)`
`await`	"Wait for this, but don't block everything"	`await client.get(url)`
`asyncio.run()`	Starts the async system	`asyncio.run(main())`
`x: str`	Type hint — "x should be a string"	Documentation only
`-> list[dict]`	Type hint — "returns a list of dicts"	Documentation only
`@dataclass`	Auto-generates `__init__` and more	Settings class
`@property`	Makes a method look like an attribute	`settings.var_confidence_step`
`field(default_factory=...)`	Lazy default value for dataclass	Delayed evaluation
`f"text {var}"`	f-string — inserts variable into text	`f"Hello {name}"`

Syntax	Meaning	Example
`date | None`	Union type — "either a date or None"	Type hint syntax
`for i in range(0, len(x), 1000):`	Loop in chunks of 1000	Batch processing
`x[i : i + 1000]`	Slice — get items i through i+999	`chunk = rows[i : i + 1000]`
`list(reversed(data))`	Reverse a list	Flip oldest→newest
`{r["ticker"] for r in rows}`	Set comprehension — unique values	Remove duplicates
`data.get("key", default)`	Safe dict access with fallback	Won't crash if key is missing

What to Do Next

1. **Run** `python main.py --describe` to see how the config looks
2. **Set up your** `.env` **file** with your actual API keys
3. **Run** `python main.py --run-daily` to see the pipeline in action
4. **Read the logs** — they'll tell you exactly what's happening at each step
5. **Change** `test_mode` **to** `False` when you're ready for production

You've got this! The code is well-structured — once you understand these patterns, you'll see them everywhere in professional Python projects.